



🌟 *The little ASGI framework that shines.* 🌟

🔄 Test Suite passing | 📦 pypi package 1.0.0 | 🐍 python 3.10 | 3.11 | 3.12 | 3.13 | 3.14 | 💬 chat 9 online

Documentation: <https://starlette.dev>

Source Code: <https://github.com/Kludex/starlette>

Introduction

Starlette is a lightweight [ASGI](#) framework/toolkit, which is ideal for building async web services in Python.

It is production-ready, and gives you the following:

- A lightweight, low-complexity HTTP web framework.
- WebSocket support.
- In-process background tasks.
- Startup and shutdown events.
- Test client built on `httpx`.
- CORS, GZip, Static Files, Streaming responses.
- Session and Cookie support.
- 100% test coverage.
- 100% type annotated codebase.
- Few hard dependencies.
- Compatible with `asyncio` and `trio` backends.
- Great overall performance [against independent benchmarks](#).

Sponsorship

Help us keep Starlette maintained and sustainable by [becoming a sponsor](#).

Current sponsors:



Hugging Face

Installation

```
pip install starlette
```

You'll also want to install an ASGI server, such as [uvicorn](#), [daphne](#), or [hypercorn](#).

```
pip install uvicorn
```

Example

main.py

```
from starlette.applications import Starlette
from starlette.responses import JSONResponse
from starlette.routing import Route

async def homepage(request):
    return JSONResponse({'hello': 'world'})

app = Starlette(debug=True, routes=[
    Route('/', homepage),
])
```

Then run the application...

```
uvicorn main:app
```

Dependencies

Starlette only requires `anyio`, and the following dependencies are optional:

- [httpx](#) - Required if you want to use the `TestClient`.
- [jinja2](#) - Required if you want to use `Jinja2Templates`.
- [python-multipart](#) - Required if you want to support form parsing, with `request.form()`.
- [itsdangerous](#) - Required for `SessionMiddleware` support.
- [pyyaml](#) - Required for `SchemaGenerator` support.

You can install all of these with `pip install starlette[full]`.

Framework or Toolkit

Starlette is designed to be used either as a complete framework, or as an ASGI toolkit. You can use any of its components independently.

main.py

```
from starlette.responses import PlainTextResponse

async def app(scope, receive, send):
    assert scope['type'] == 'http'
    response = PlainTextResponse('Hello, world!')
    await response(scope, receive, send)
```

Run the app application in main.py:

```
$ uvicorn main:app
INFO: Started server process [11509]
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Run uvicorn with `--reload` to enable auto-reloading on code changes.

Modularity

The modularity that Starlette is designed on promotes building re-usable components that can be shared between any ASGI framework. This should enable an ecosystem of shared middleware and mountable applications.

The clean API separation also means it's easier to understand each component in isolation.

Starlette is [BSD licensed](#) code.
Designed & crafted with care.

— ★ —

Made with [Zensical](#)

